CyberTrack

Cyber Incident Tracking System

RDBMS Project Demo File

Project Name	CyberTrack
Student Name	Waris Sharma
Course	BCA Cybersecurity
Semester	3rd
Subject	RDBMS
Roll Number	25202010053
Database	cybertrack (MySQL / MySQL Workbench)

1. Introduction

CyberTrack is a relational database project built in MySQL to record, classify and track cybersecurity incidents in an organisation. It stores who reported an incident, its type and severity, which analyst investigated it, what digital evidence was collected, and how its status changed over time. This file demonstrates the working implementation through SQL queries executed in MySQL Workbench, each followed by its output screenshot.

2. Objectives

• Store incident data in a normalised relational structure with primary and foreign keys.
• Demonstrate JOINs, aggregate functions, GROUP BY / HAVING, subqueries, UNION and indexes.
• Demonstrate transactions with SAVEPOINT, ROLLBACK and COMMIT.
• Maintain an automatic audit trail of status changes using a trigger.

3. Database Design

The schema **cybertrack** contains seven tables:

Table	Purpose	Key columns
users	Reporters and security analysts	user_id (PK), full_name
incident_types	Incident categories (Phishing, Malware...)	type_id (PK), type_name, description
severity	Low, Medium, High, Critical	severity_id (PK), severity_name, priority_score
incidents	Main table of reported incidents	incident_id (PK), title, description, type_id, severity_id, reported_by (FKs), status, incident_date, resolved_date
investigations	Investigation work on incidents	investigation_id (PK), incident_id, investigator_id (FKs), investigation_notes, investigation_date, resolution
evidence	Evidence files for incidents	incident_id (FK), file_name, file_size_kb
incident_status_history	Audit log of status changes	history_id (PK), incident_id, old_status, new_status, changed_by, changed_at, remarks

Relationships: incident type → many incidents; severity → many incidents; user → many reported incidents; incident → many investigations, evidence files and status-history records; user (analyst) → many investigations.

4. Implementation and Output

4.1 Investigation Details (INNER JOIN)

Aim: Show every investigation with its incident title and investigator name.

```
SELECT INV.investigation_id,
       I.title AS incident_title,
       U.full_name AS investigator,
       INV.investigation_notes,
       INV.investigation_date,
       INV.resolution
FROM Investigations INV
INNER JOIN Incidents I ON INV.incident_id = I.incident_id
INNER JOIN Users U ON INV.investigator_id = U.user_id;
```

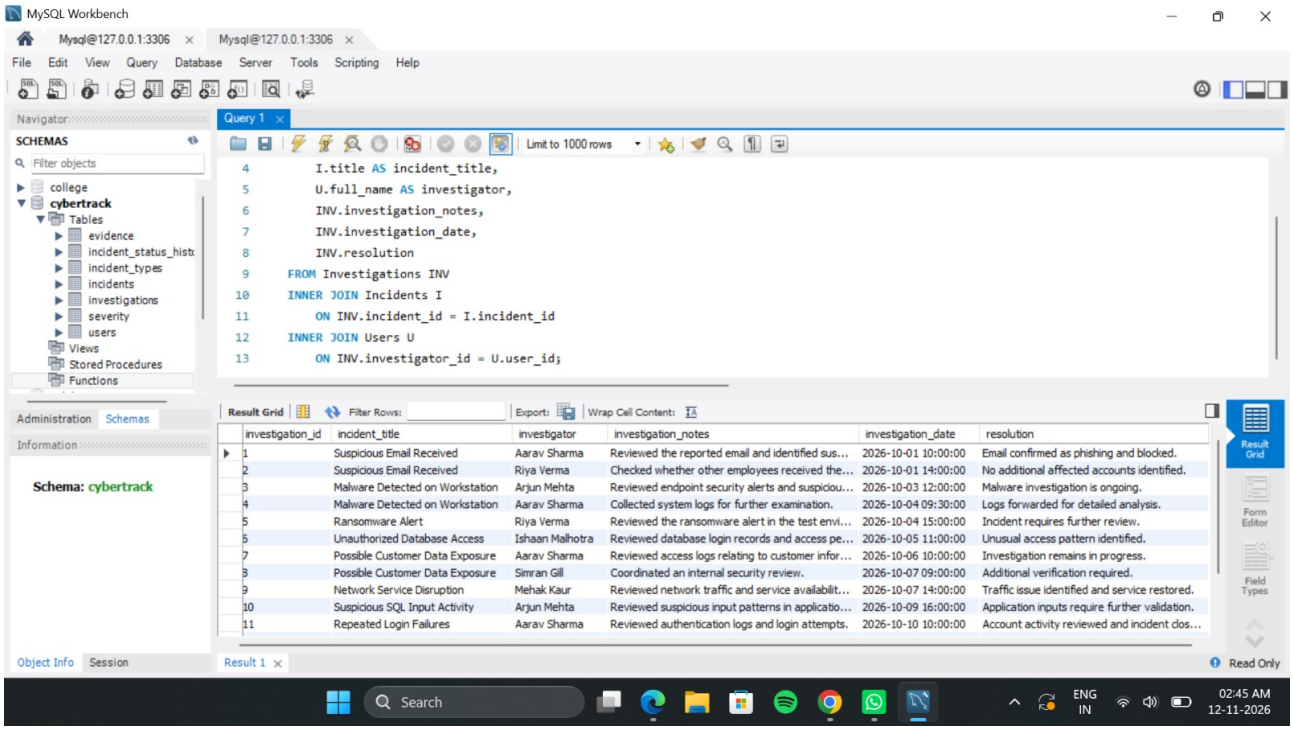


Figure 1: Result 1 - investigations with incident and investigator

Output: Each investigation is listed with its incident (e.g. Suspicious Email Received investigated by Aarav Sharma and Riya Verma) and its resolution.

4.2 Complete Incident Report (Multiple JOINs)

Aim: Combine incident, type, severity, reporter, investigator, status and date in one report.

```
SELECT ...
FROM Incidents I
INNER JOIN Incident_Types T ON I.type_id = T.type_id
INNER JOIN Severity S ON I.severity_id = S.severity_id
INNER JOIN Users U ON I.reported_by = U.user_id
LEFT JOIN Investigations INV ON I.incident_id = INV.incident_id
LEFT JOIN Users INV_USER ON INV.investigator_id = INV_USER.user_id
ORDER BY I.incident_id;
```

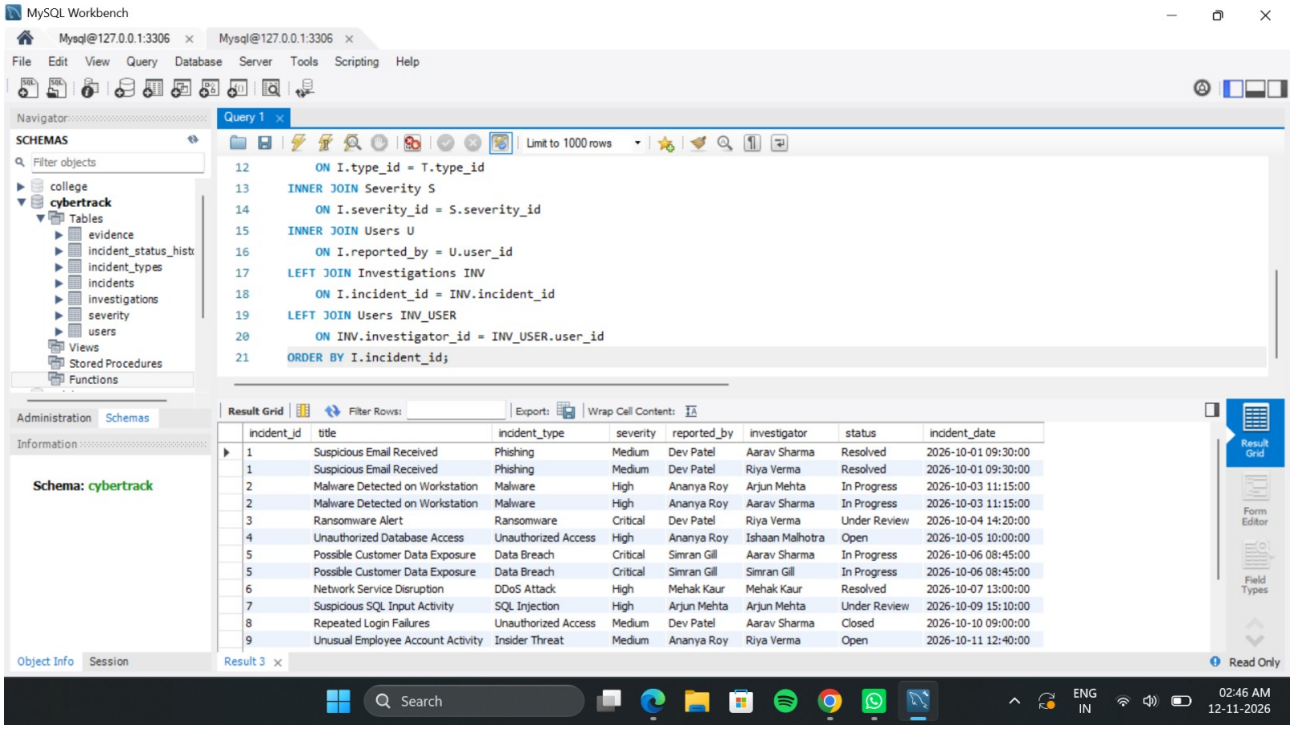


Figure 2: Result 3 - full incident report

Output: One row per incident-investigator pair. LEFT JOIN keeps incidents even without an investigation; incidents with two investigators (1, 2, 5) appear twice.

4.3 Status History of Incident 12

Aim: View the audit trail of incident 12 in chronological order.

```
SELECT *
FROM Incident_Status_History
WHERE incident_id = 12
ORDER BY changed_at;
```

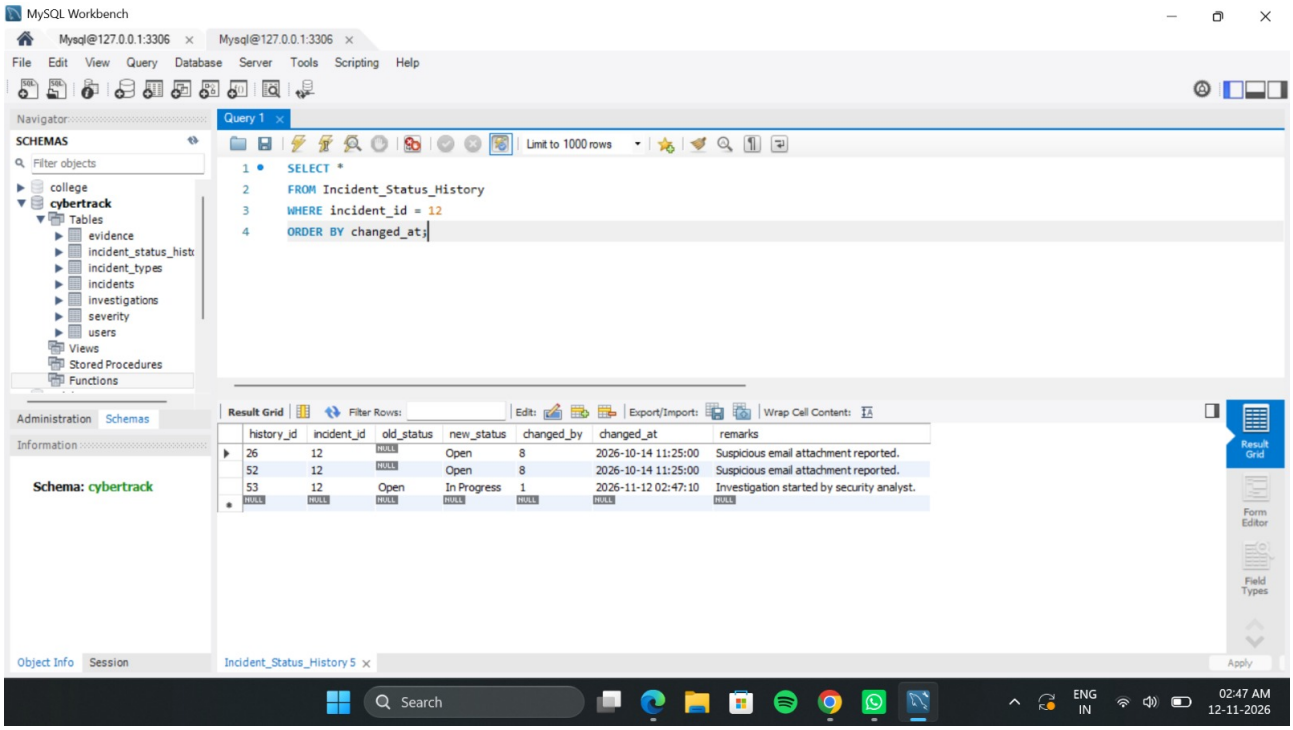


Figure 3: Incident_Status_History for incident 12

Output: Incident 12 was recorded as Open on 2026-10-14 and later moved from Open to In Progress with the remark 'Investigation started by security analyst.'

4.4 Incidents with Severity and Reporter

Aim: List incidents with severity, reporter name and status.

```
SELECT I.incident_id, I.title, S.severity_name,
       U.full_name AS reported_by, I.status
FROM incidents I
JOIN severity S ON I.severity_id = S.severity_id
JOIN users U ON I.reported_by = U.user_id;
```

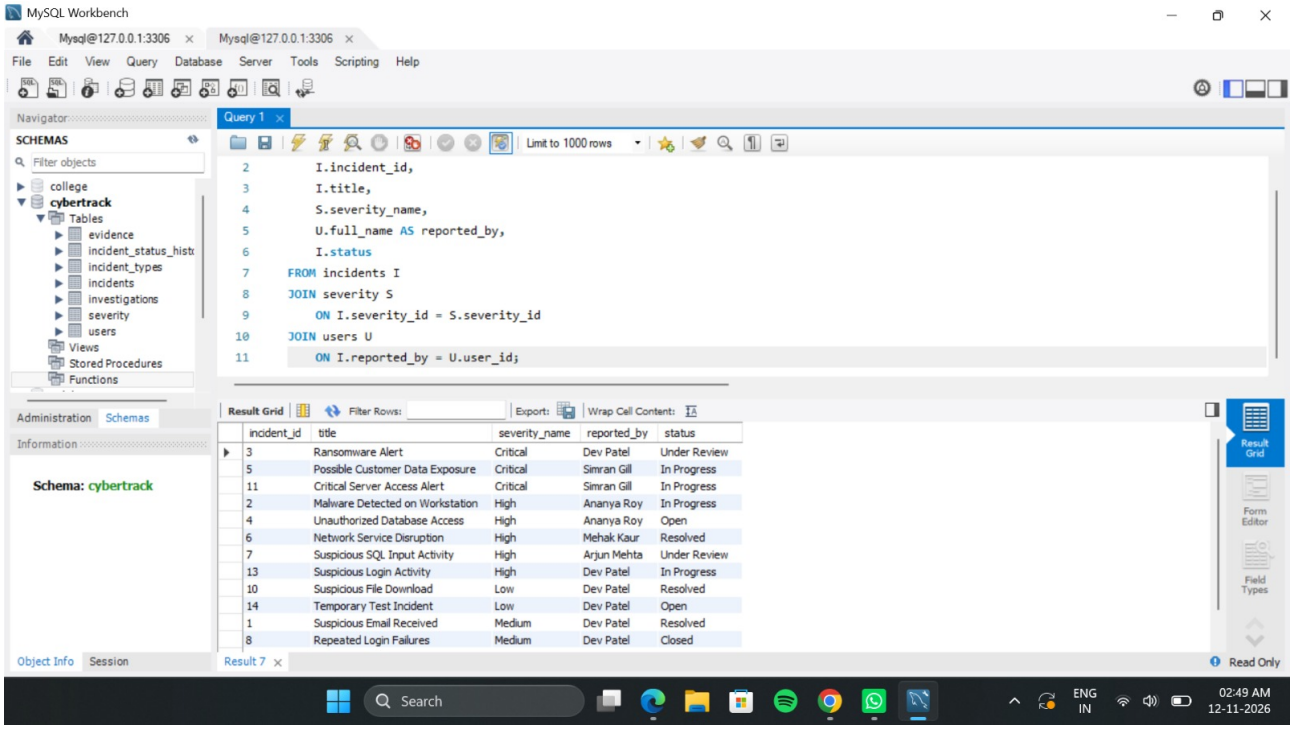


Figure 4: Result 7 - incidents with severity and reporter

Output: Incidents appear ordered by severity, e.g. Critical: Ransomware Alert (Dev Patel, Under Review), Possible Customer Data Exposure (Simran Gill, In Progress).

4.5 Incident Count by Severity (GROUP BY)

Aim: Count incidents per severity level.

```
SELECT S.severity_name, COUNT(I.incident_id) AS total_incidents
FROM incidents I
JOIN severity S ON I.severity_id = S.severity_id
GROUP BY S.severity_name;
```

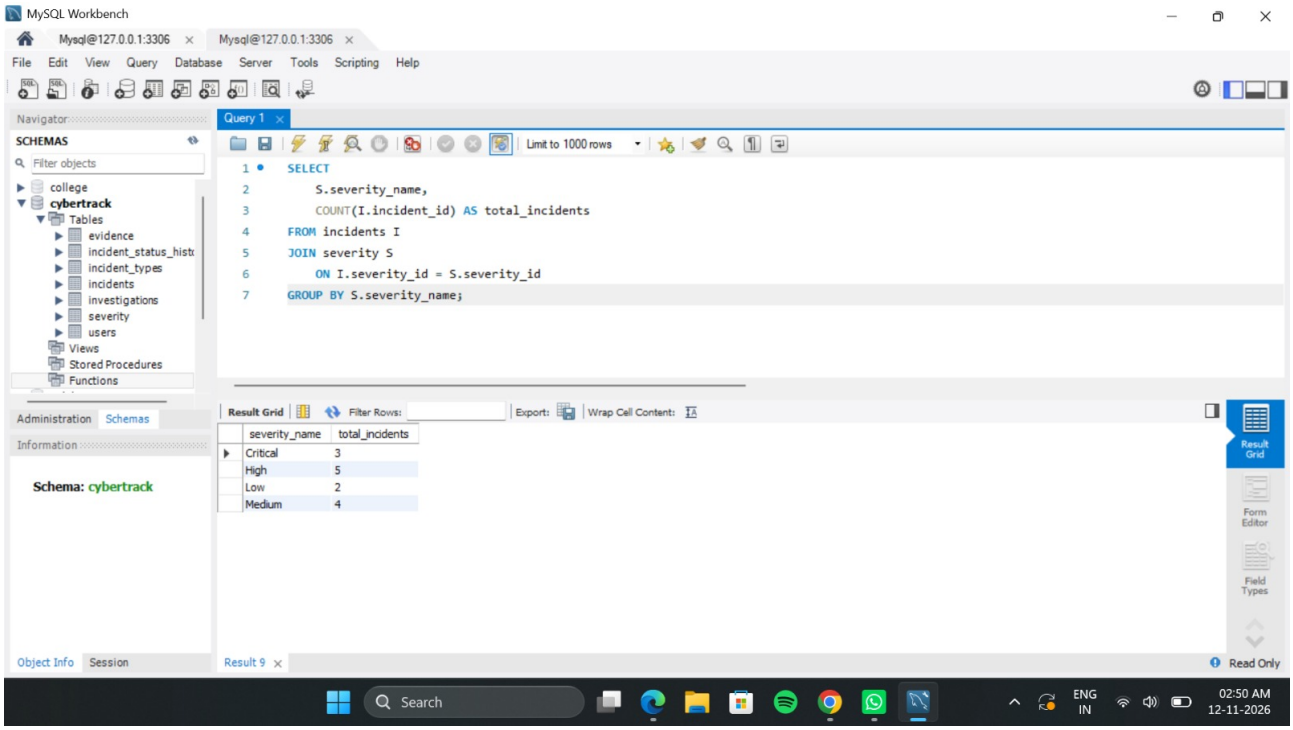


Figure 5: Result 9 - incidents grouped by severity

Output: Critical = 3, High = 5, Low = 2, Medium = 4.

4.6 Total Evidence Size (SUM)

Aim: Find the total size of all evidence files.

```
SELECT SUM(file_size_kb) AS total_evidence_size_kb
FROM evidence;
```

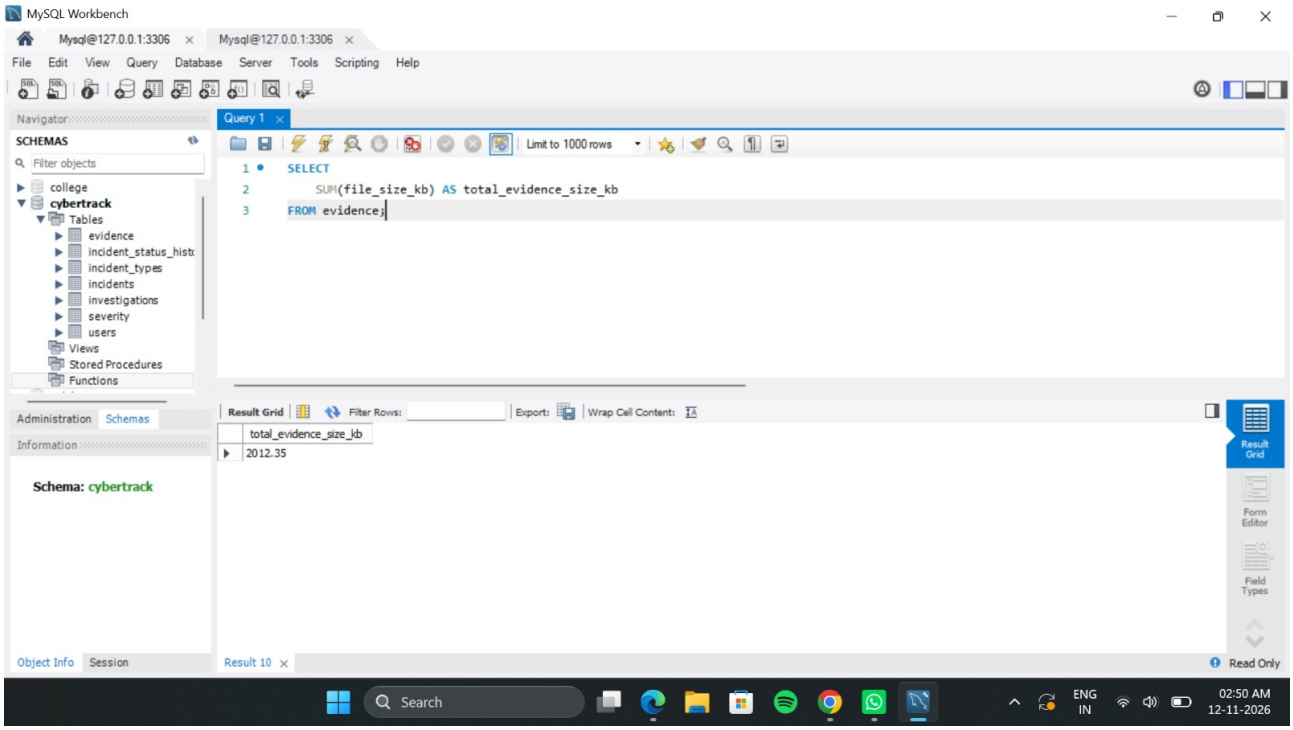


Figure 6: Result 10 - SUM of evidence size

Output: total_evidence_size_kb = 2012.35 KB.

4.7 Smallest and Largest Evidence File (MIN, MAX)

Aim: Find the smallest and largest evidence file sizes.

```
SELECT MIN(file_size_kb) AS smallest_file_kb,
       MAX(file_size_kb) AS largest_file_kb
FROM evidence;
```

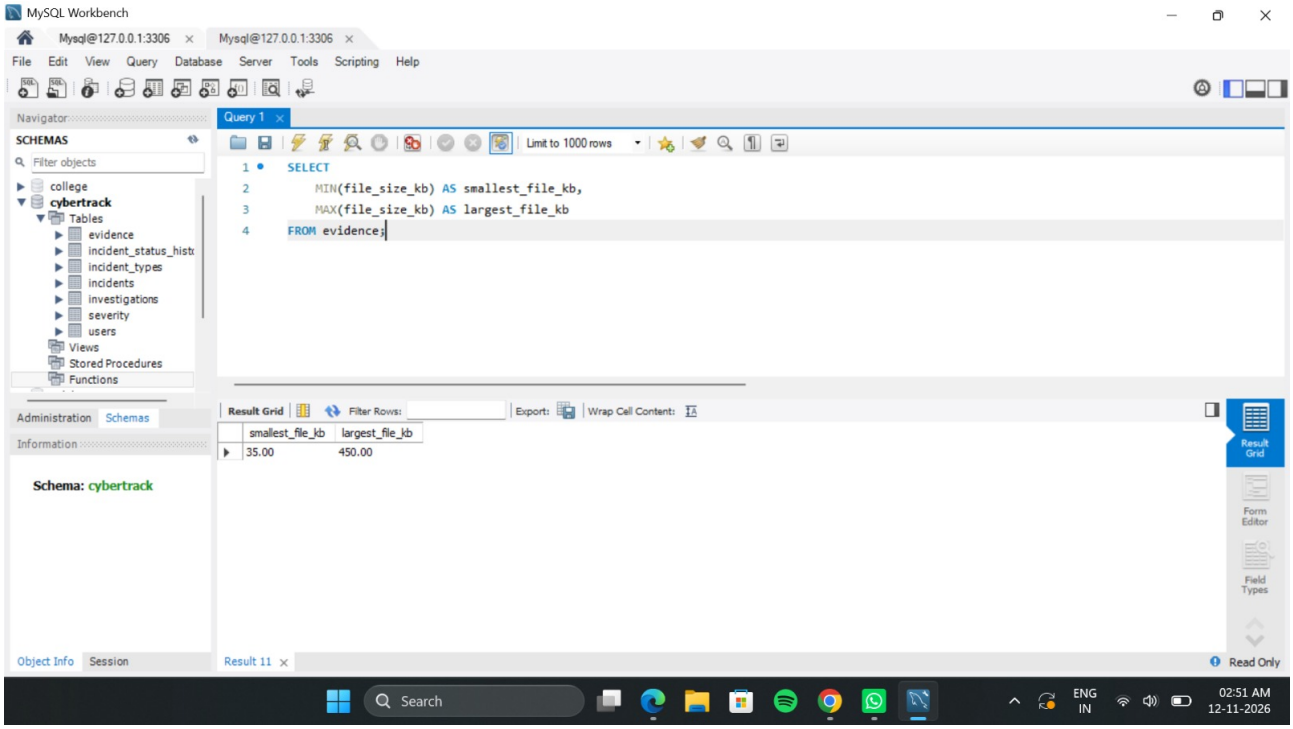


Figure 7: Result 11 - MIN and MAX of evidence size

Output: Smallest file = 35.00 KB; largest file = 450.00 KB.

4.8 Incident Types with More Than One Incident (HAVING)

Aim: Show only incident types that occur more than once.

```
SELECT T.type_name, COUNT(I.incident_id) AS total_incidents
FROM incident_types T
JOIN incidents I ON T.type_id = I.type_id
GROUP BY T.type_name
HAVING COUNT(I.incident_id) > 1;
```

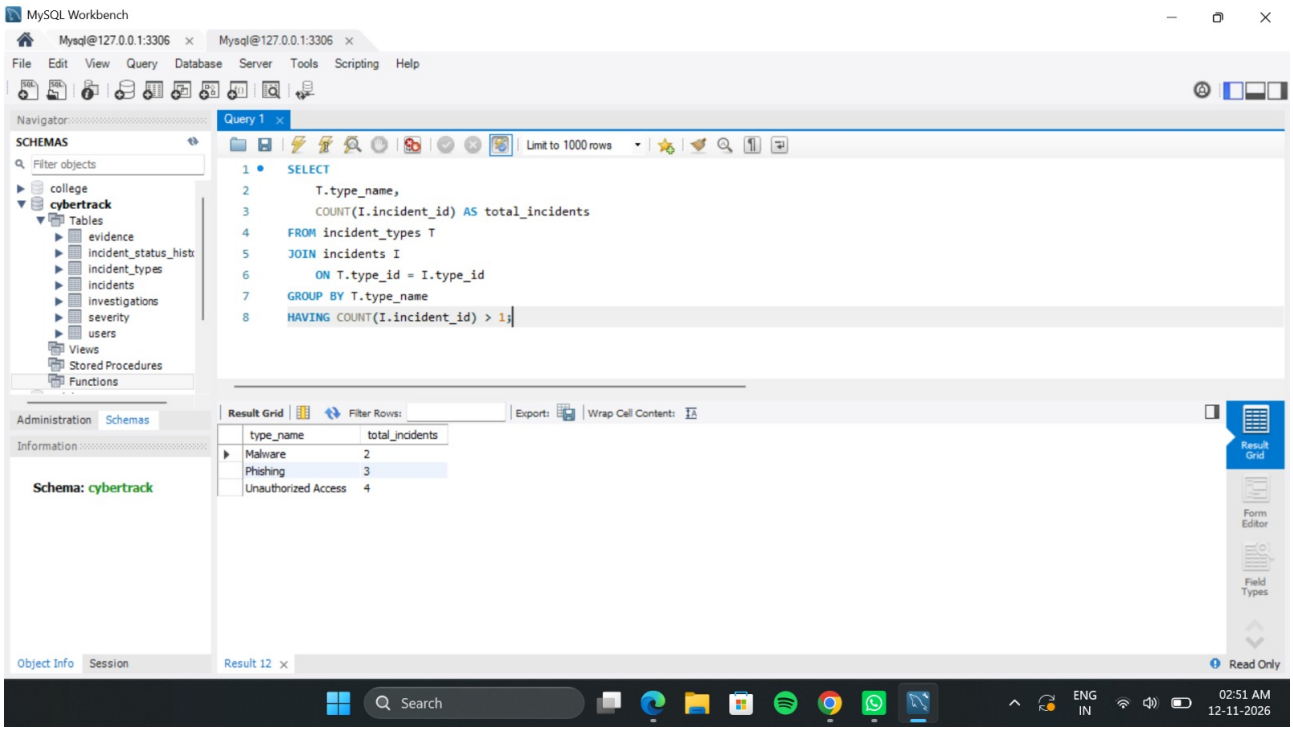


Figure 8: Result 12 - GROUP BY with HAVING

Output: Malware = 2, Phishing = 3, Unauthorized Access = 4. HAVING filters groups after aggregation.

4.9 Subquery - Critical Incidents

Aim: Retrieve incidents whose severity is 'Critical'.

```
SELECT *
FROM incidents
WHERE severity_id = (
    SELECT severity_id
    FROM severity
    WHERE severity_name = 'Critical'
);
```

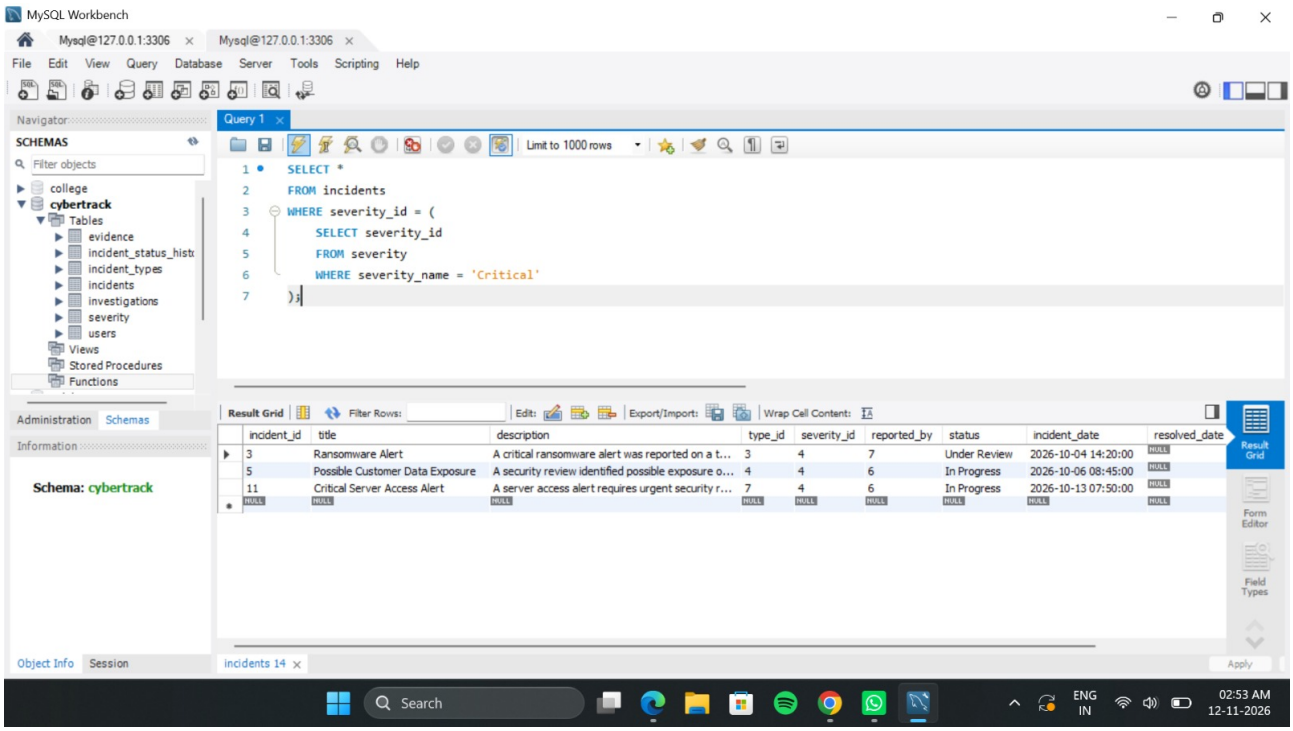


Figure 9: Subquery for Critical severity

Output: Incidents 3 (Ransomware Alert), 5 (Possible Customer Data Exposure) and 11 (Critical Server Access Alert).

4.10 Subquery - Above-Average Priority Score

Aim: List incidents whose severity priority score is above the average.

```
SELECT I.incident_id, I.title, S.severity_name, S.priority_score
FROM incidents I
JOIN severity S ON I.severity_id = S.severity_id
WHERE S.priority_score > (
    SELECT AVG(priority_score)
    FROM severity
);
```

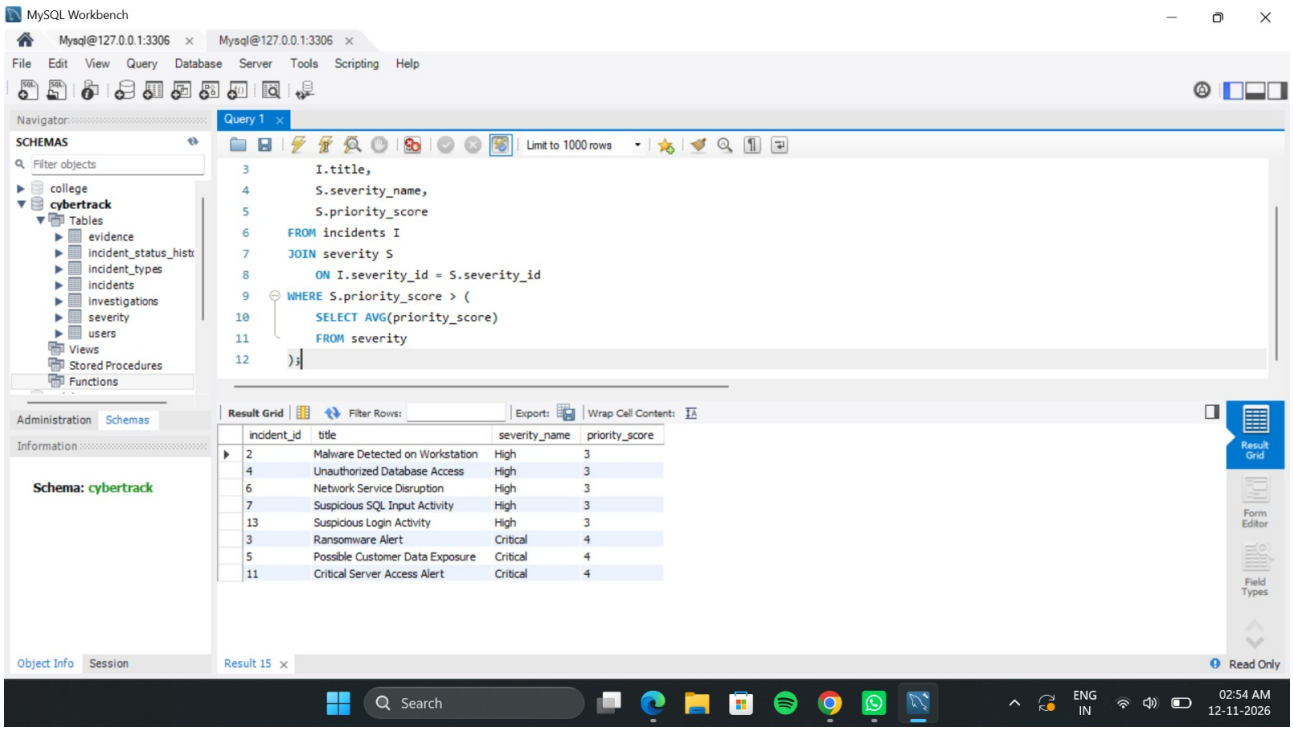


Figure 10: Result 15 - priority score above average

Output: Only High (score 3) and Critical (score 4) incidents are returned, e.g. incidents 2, 4, 6, 7, 13 and 3, 5, 11.

4.11 Correlated Subquery - Evidence Above Incident Average

Aim: Find evidence files larger than the average file size of their own incident.

```
SELECT I.incident_id, I.title, E.file_name, E.file_size_kb
FROM incidents I
JOIN evidence E ON I.incident_id = E.incident_id
WHERE E.file_size_kb > (
    SELECT AVG(E2.file_size_kb)
    FROM evidence E2
    WHERE E2.incident_id = I.incident_id
);
```

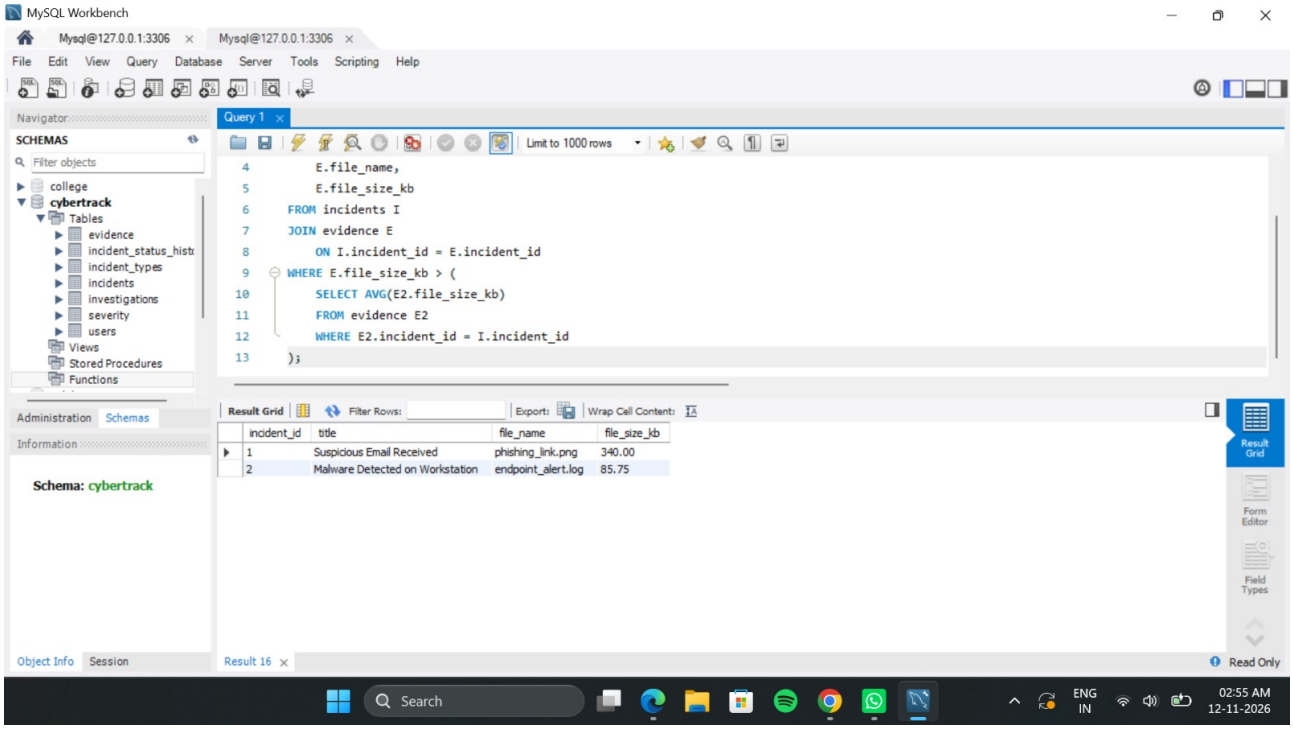


Figure 11: Result 16 - correlated subquery

Output: phishing_link.png (340.00 KB) for incident 1 and endpoint_alert.log (85.75 KB) for incident 2. The inner query runs once per incident.

4.12 Indexes on the Incidents Table

Aim: Display the indexes defined on the incidents table.

```
SHOW INDEX FROM incidents;
```

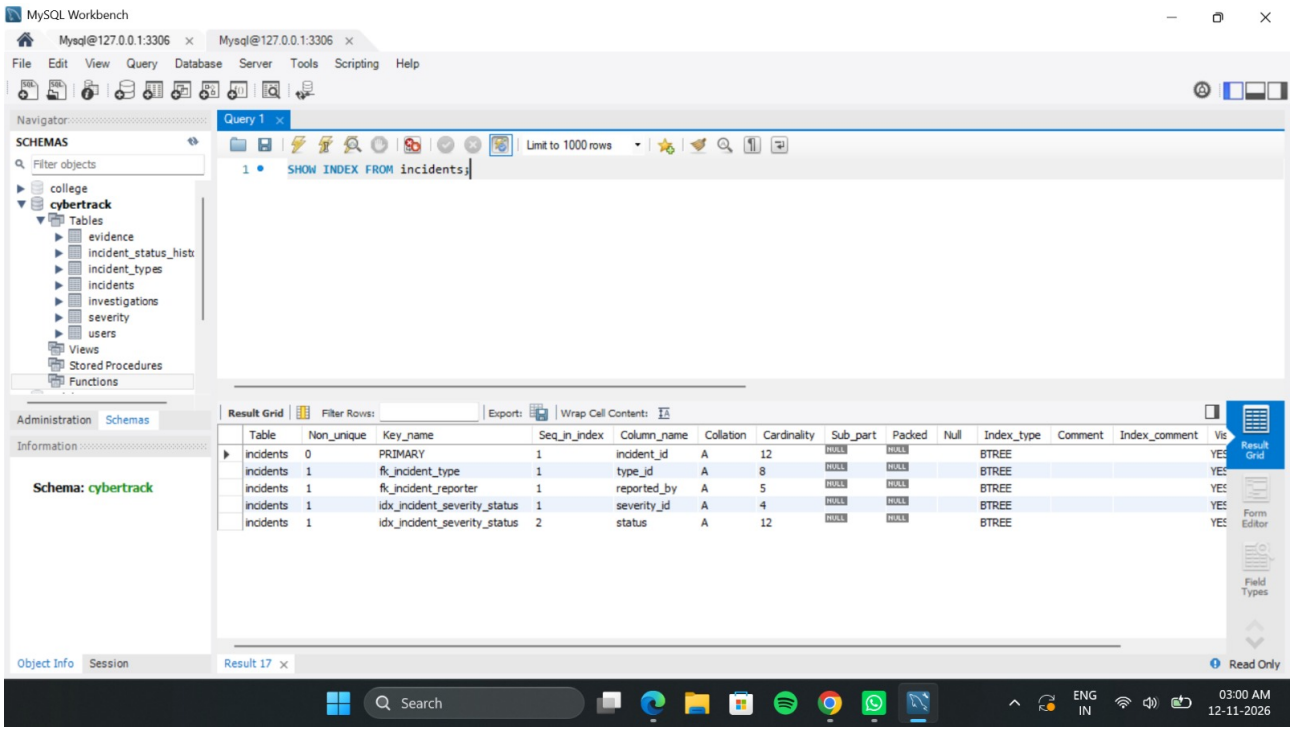


Figure 12: Result 17 - SHOW INDEX

Output: PRIMARY (incident_id), fk_incident_type (type_id), fk_incident_reporter (reported_by) and a composite index idx_incident_severity_status (severity_id, status), all BTREE. The composite index speeds up searches by severity and status together.

4.13 Transaction with ROLLBACK

Aim: Show that changes can be undone before they are saved.

```
START TRANSACTION;

UPDATE incidents
SET status = 'Under Review'
WHERE incident_id = 12;

SELECT * FROM incidents
WHERE incident_id = 12;

ROLLBACK;
```

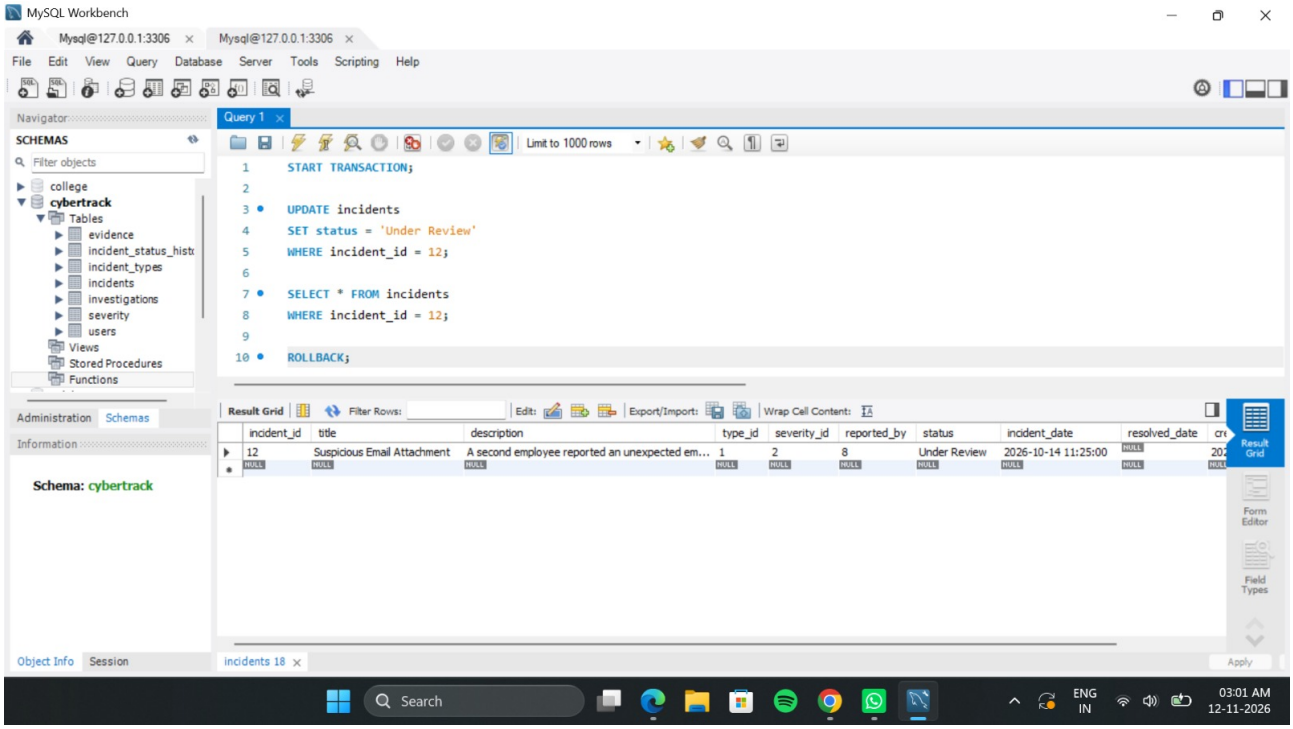


Figure 13: Transaction with ROLLBACK

Output: Inside the transaction incident 12 shows status 'Under Review'. After ROLLBACK the original status is restored and nothing is saved permanently.

4.14 SAVEPOINT, Partial ROLLBACK and COMMIT

Aim: Undo only part of a transaction.

```
-- Test Type A inserted, then savepoint_one created
-- Test Type B inserted
ROLLBACK TO SAVEPOINT savepoint_one;

COMMIT;
SELECT *
FROM incident_types
WHERE type_name IN ('Test Type A', 'Test Type B');
```

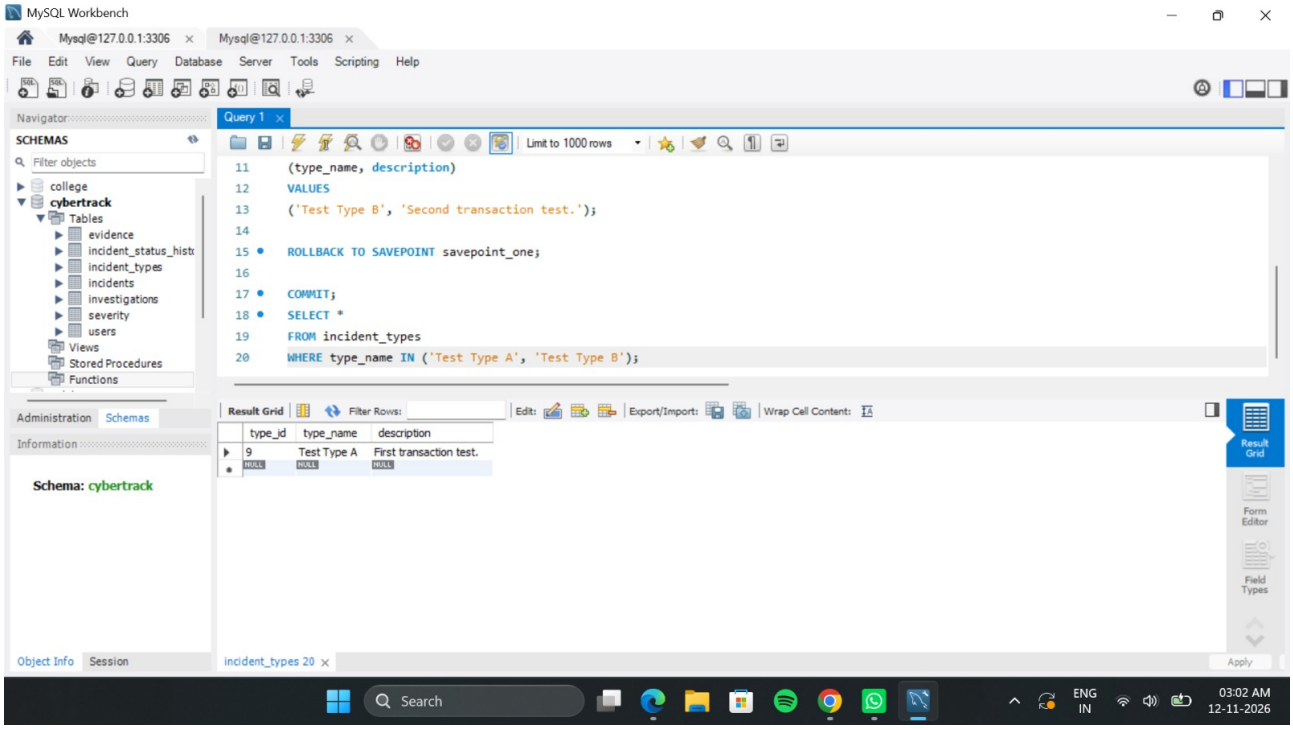


Figure 14: SAVEPOINT and ROLLBACK TO SAVEPOINT

Output: Only 'Test Type A' (type_id 9) exists. 'Test Type B' was rolled back to the savepoint, while Test Type A was saved by COMMIT.

4.15 Automatic Status Logging (Trigger)

Aim: Check the audit trail of incident 4, newest first.

```
SELECT *
FROM incident_status_history
WHERE incident_id = 4
ORDER BY changed_at DESC;
```

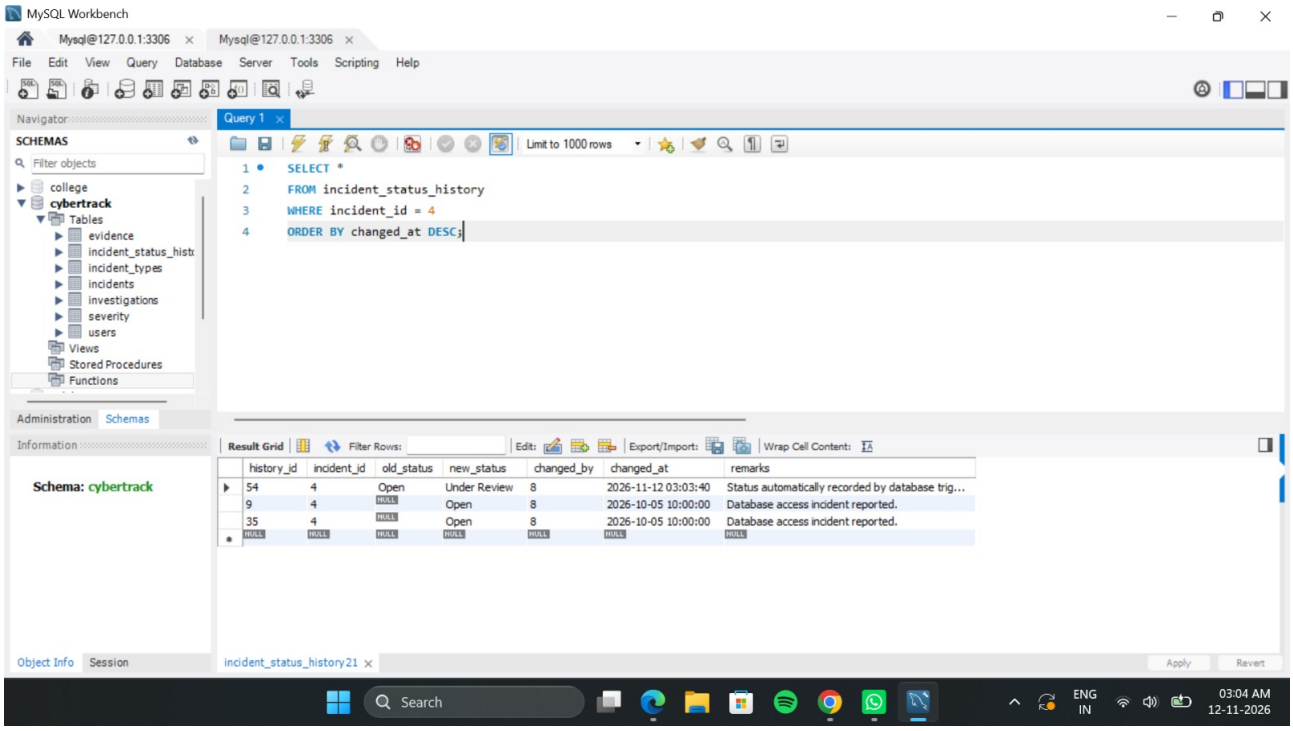


Figure 15: Trigger-generated status history

Output: The newest record (history_id 54) shows incident 4 changing from Open to Under Review with the remark 'Status automatically recorded by database trigger.' This confirms the trigger logs each status update.

4.16 UNION ALL

Aim: Combine two result sets of user names, keeping duplicates.

```
SELECT full_name AS person_name
FROM users
WHERE user_id <= 5

UNION ALL

SELECT full_name AS person_name
FROM users
WHERE user_id >= 5;
```

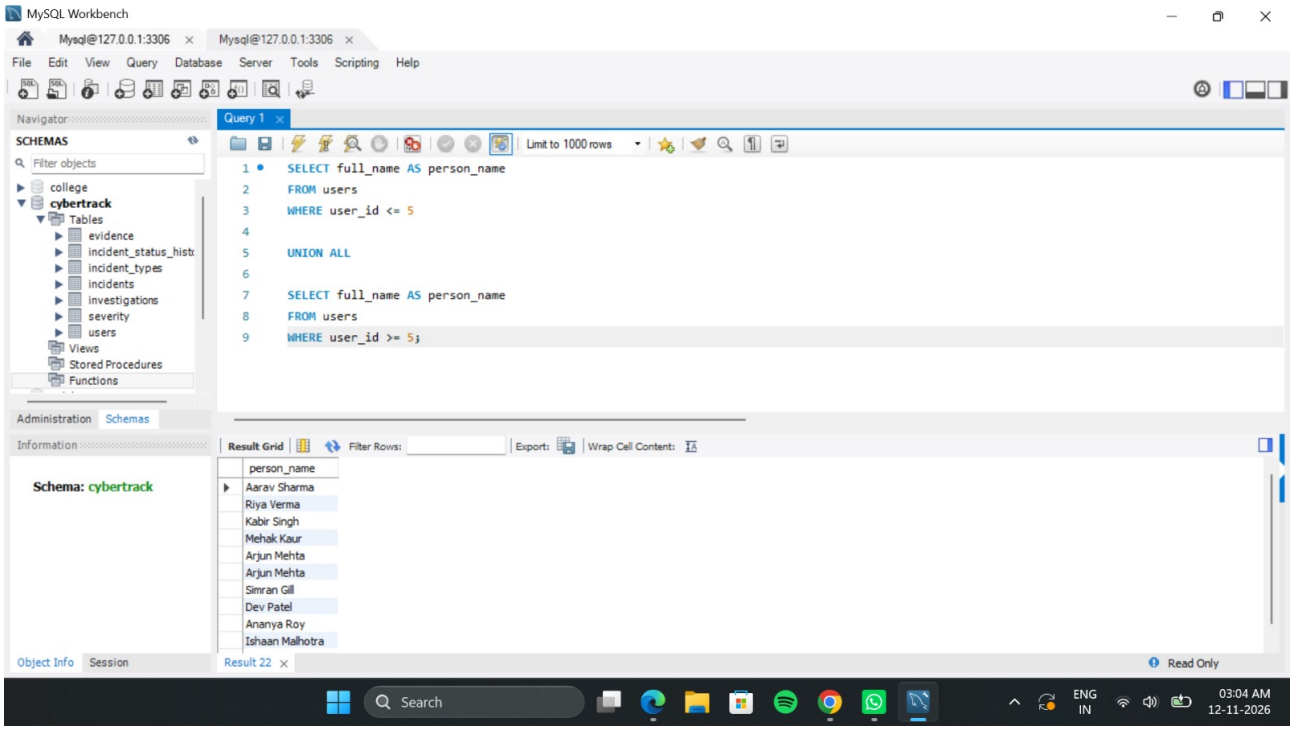


Figure 16: Result 22 - UNION ALL

Output: Both result sets are combined. User 5 (Arjun Mehta) matches both conditions, so the name appears twice. UNION ALL keeps duplicates; UNION would remove them.

5. Concepts Demonstrated

Concept	Sections
INNER JOIN / LEFT JOIN	4.1, 4.2, 4.4
Aggregate functions (COUNT, SUM, MIN, MAX, AVG)	4.5 - 4.7, 4.10, 4.11
GROUP BY and HAVING	4.5, 4.8
Subqueries (single-value and correlated)	4.9 - 4.11
Indexes	4.12
Transactions, SAVEPOINT, ROLLBACK, COMMIT	4.13, 4.14
Triggers and audit history	4.3, 4.15
UNION ALL	4.16

6. Conclusion

The CyberTrack database manages the full lifecycle of a cybersecurity incident: reporting, classification, investigation, evidence collection and status tracking. The queries show a properly related schema, useful reports through joins and aggregates, better performance through indexes, data safety through transactions, and an automatic audit trail through triggers.

Submitted by: Waris Sharma | BCA Cybersecurity | Semester 3 | Roll No. 25202010053